

Utilizando React, este proyecto se beneficia de una biblioteca avanzada para aplicaciones web, ideal para crear una experiencia de usuario fluida y de alta interactividad en entornos virtuales.

React se destaca por su capacidad de construir interfaces de usuario eficientes y responsivas, lo que es crucial para cualquier plataforma que busca ofrecer una experiencia de usuario óptima. Su enfoque en la declaratividad y el renderizado eficiente asegura que los usuarios experimenten transiciones suaves y una interacción continua, sin demoras, al navegar por diferentes escenarios.

TypeScript para un desarrollo sólido: La implementación de TypeScript mejora la calidad del código y facilita la detección de errores, lo que es fundamental en el desarrollo de aplicaciones complejas. Esta combinación permite un desarrollo más robusto y mantenible, asegurando que la complejidad inherente a aplicaciones avanzadas se maneje de manera eficiente.

Tailwind CSS para una interfaz intuitiva y adaptativa: El uso de Tailwind CSS asegura una interfaz de usuario coherente y visualmente atractiva, lo que es vital para mantener a los usuarios comprometidos. Su diseño responsivo se adapta perfectamente a diferentes dispositivos, mejorando la accesibilidad y la experiencia de usuario en cualquier aplicación.

Vite.js para un desarrollo y construcción eficientes: Vite.js, seleccionado por su velocidad y eficiencia, se integra con TypeScript para proporcionar un entorno de desarrollo ágil. Esto permite un ciclo de desarrollo más rápido, con tiempos de carga instantáneos y feedback rápido para los desarrolladores, lo que es crucial para iterar rápidamente en el diseño y funcionalidad de cualquier plataforma.

Zustand para una gestión de estado eficiente: Zustand, seleccionado por su simplicidad y flexibilidad, se integra con React y TypeScript para proporcionar una gestión de estado ágil. Esto permite un manejo del estado global más intuitivo, con una API mínima y sin complicaciones para los desarrolladores, lo que es significativo para mantener la claridad y eficiencia en la lógica de estado de cualquier aplicación.

Yarn para la gestión eficiente de paquetes: La adopción de Yarn mejora el proceso de desarrollo al optimizar el almacenamiento y el rendimiento de la instalación de dependencias. Esto facilita el mantenimiento y la escalabilidad del proyecto, asegurando una base de código limpia y una integración sin problemas de nuevas tecnologías o bibliotecas.

[Visitando este enlace podrás tener acceso a todas las imágenes del proyecto](#)

Fragmento de código en donde se aprecia la estructura de manejo de errores en el frontend.

```
1 const createErrorFactory = (name: string, message?: string) => {
2   return class CustomizedError extends Error {
3     constructor(error?: Error) {
4       super(error?.message || message)
5       this.name = name
6       this.stack = ''
7     }
8   }
9 }
10
11 export const ConnectionError = createErrorFactory(
12   'ConnectionError',
13   'Conexión fallida. No se pudo establecer conexión con el servidor' "Conexión": Unknown word.
14 )
15 export const ValidationError = createErrorFactory('ValidationError')
16 export const AuthenticationError = createErrorFactory('AuthenticationError', 'Email o contraseña inválidos') "contraseña": Unknown word.
17 export const AuthorizationError = createErrorFactory('AuthorizationError', 'No tiene permisos suficientes') "tiene": Unknown word.
18 export const NotFoundError = createErrorFactory('NotFoundError', 'Página no encontrada') "Página": Unknown word.
19 export const TimeoutError = createErrorFactory('TimeoutError', 'Error de timeout. Intente de nuevo más tarde') "Intente": Unknown word.
20 export const ConflictError = createErrorFactory('ConflictError', 'Ya existe un usuario con esas credenciales') "existe": Unknown word.
21 export const ServerError = createErrorFactory('ServerError')
22 export const ClientError = createErrorFactory(
23   'ClientError',
24   'Parse recibió un JSON inválido. Puede haber errores de sintaxis, como comillas mal colocadas, comas faltantes, o falta de llaves de apertura o c
25 )
26 export const DatabaseError = createErrorFactory('DatabaseError')
27
```

[Ver imagen](#)

```
1 import {
2   ConnectionError,
3   ValidationError,
4   AuthenticationError,
5   AuthorizationError,
6   NotFoundError,
7   TimeoutError,
8   ConflictError,
9   ServerError,
10  ClientError,
11  DatabaseError,
12 } from './factory'
13
14 const errorHandler = (error: unknown) => {
15   if (error instanceof Error) {
16     if (error.message === 'XMLHttpRequest failed: "Unable to connect to the Parse API"') {
17       throw new ConnectionError()
18     }
19     if (error.message.includes('XMLHttpRequest failed:')) {
20       throw new ConnectionError(error)
21     }
22     if (error.message.includes('Received an error with invalid JSON from Parse:')) {
23       throw new ServerError(error)
24     }
25     if (error.message === '') {
26       throw new ValidationError()
27     }
28   }
29 }
```

[Ver imagen](#)

Fragmento de código en donde se aprecia el manejo encapsulado de Hooks, en este caso las imágenes muestran los hooks que usamos para el inicio de sesión del usuario.

```
import { Moralis } from "moralis-v1";
import { useState } from "react";
import type { LogIn } from "../types";
import handleError from "@errors";

function useLogIn() {
  const [loading, setLoading] = useState(false);

  const logIn = async ({ email, password }: LogIn) => {
    setLoading(true);

    try {
      const response = await Moralis.User.logIn(email, password);
      if (response.id) {
        return { success: "Registro exitoso" };
      }
    } catch (error) {
      return handleError(error);
    } finally {
      setLoading(false);
    }
  };

  return {
    logIn,
    loading,
  };
}

export default useLogIn;
```

En la siguiente imagen se muestra el componente con el formulario de autenticación de usuario

```

12
13 export default function LogInForm() {
14   const { logIn, loading } = useLogIn()
15   const navigate = useNavigate()
16   const { showBoundary } = useErrorBoundary()
17   const formik = useFormik({
18     initialValues: {
19       email: '',
20       password: '',
21     } as LogIn,
22     validationSchema: LogInSchema,
23     onSubmit: async ({ email, password }) => {
24       try {
25         const result = await logIn({
26           email,
27           password,
28         })
29         if (result?.success) navigate(Routes.home)
30       } catch (error) {
31         return showBoundary(error)
32       }
33     },
34   })
35
36   return (
37     <main className="">
38       <div className="">
39         <span className="">Iniciar sesión</span>
40       </div>
41       <form className="" onSubmit={formik.handleSubmit}>
42         </* Input Email */>
43         <label className="">
44           <span>Correo</span>
45           <input
46             onChange={formik.handleChange}
47             value={formik.values.email}
48             onBlur={formik.handleBlur}
49             name="email"
50             placeholder="email@gmail.com"

```

[Ver imagen](#)

En el siguiente fragmento de código mostramos el uso de la biblioteca yup para definir esquemas de validación para formularios de registro e inicio de sesión.

```

import * as yup from 'yup'

const passwordRules =
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#.%&@\$\%^&\*])(?=.{8,})/
const usernameSignUp = /^(S+$)/g
const emailRules = /^[^@]+@^[^@]+\.[^@]+$

export const SignUpSchema = yup.object().shape({
  fullName: yup
    .string()

```

```

    .min(5, 'Full Name must be at least 5 characters long')
    .max(65, 'Full name must contain a maximum of 65 characters')
    .required('Required, Please Enter your Full Name'),
  username: yup
    .string()
    .min(5, 'Username must be at least 5 characters long')
    .max(25, 'User name must contain a maximum of 25 characters')
    .required('Required, Please Enter your User Name ')
    .matches(usernameSignUp, 'spaces not allowed'),
  email: yup.string().max(255).email('Must be a valid email').required('Email
is required'),
  password: yup
    .string()
    .required('Please Enter your password')
    .matches(
      passwordRules,
      'Must Contain 8 Characters, One Uppercase, One Lowercase, One Number and
One Special Case Character: ! @ # . * % & @'
    ),
}))

export const LogInSchema = yup.object().shape({
  email: yup
    .string()
    .min(5, 'El nombre de usuario debe contener al menos 5 caracteres')
    .max(25, 'El nombre de usuario debe tener un máximo de 25 caracteres')
    .required('Requerido, por favor introduzca su nombre de usuario')
    .matches(emailRules, 'Debe ser un mail'),
  password: yup
    .string()
    .required('Requerido, por favor ingrese la contraseña')
    .matches(
      passwordRules,
      'Debe contener 8 caracteres, una mayúscula, una minúscula, un numero y
un carácter especial: : ! @ # . * % & @'
    ),
}))

```

En la siguiente imagen podemos apreciar lo que es el core de la aplicación, en este módulo se manejan todos los componentes, hooks y utilidades que se reutilizan en diferentes partes de la aplicación.

```
src
├── layout
├── logging
├── mockData
├── modules
│   ├── auth
│   ├── core
│   │   ├── components
│   │   │   ├── avatar
│   │   │   ├── header
│   │   │   └── footer.tsx 4
│   │   ├── pagination.tsx
│   │   ├── search.tsx 1
│   │   ├── enum
│   │   ├── hooks
│   │   │   ├── useSession.tsx
│   │   ├── icons
│   │   │   ├── arrow.tsx
│   │   │   ├── index.ts
│   │   │   ├── Loader.tsx
│   │   │   ├── logout.tsx
│   │   │   ├── person.tsx
│   │   │   ├── search.tsx
│   │   ├── lib
│   │   ├── ui
│   │   ├── utils
│   │   ├── users
│   │   ├── routes
│   │   ├── screens
│   │   ├── services
│   │   └── store
```

```
1 import { Routes } from '@routes'
2 import { Link } from 'react-router-dom'
3
4 export function Footer() {
5     return (
6         <footer className="...">
7             <div className="...">
8                 <div>
9                     <a className="..." href={Routes.home} aria-label="Brand">
10                         Brand
11                     </a>
12                 </div>
13
14                 <div className="...">
15                     <p className="...">
16                         We're part of the{' '}
17                         <a className="..." href="/">
18                             Htmlstream
19                         </a>{' '}
20                         family.
21                     </p>
22                 </div>
23
24                 <div className="...">
25                     <a
26                         className="..."
27                         href="/"
28                     >
29                         <svg
30                             className="..."
31                             xmlns="http://www.w3.org/2000/svg"
32                             width="16"
33                             height="16"
34                             fill="currentColor"
35                             viewBox="0 0 16 16"
```

[Ver imagen](#)

En el siguiente código se muestra un componente reutilizable que está dentro del core.

```
import { useNavigate } from 'react-router-dom'
import { Search as SearchIcon } from '@modules/core/icons'

export default function Search() {
    const navigate = useNavigate()
    const url = new URL(globalThis.location.href)

    const handlerSubmit = (event: React.FormEvent<HTMLFormElement>) => {
        event.preventDefault()

        const form = new FormData(event.currentTarget)

        const q = form.get('q') as string
```

```

    console.log(q)

    q?.length > 0 ? url.searchParams.set('q', q) :
url.searchParams.delete('q')

    navigate(url.pathname + url.search)
  }

  return (
    <form
      onSubmit={(e) => handlerSubmit(e)}
      className="bg-white shadow-lg border rounded-md max-w-lg w-full flex
gap-1 items-center py-1 pr-1 pl-3"
    >
      <input
        placeholder="Buscar"
        type="search"
        className="bg-transparent outline-none w-full"
        defaultValue={String(url.searchParams.get('q') || '')}
        name="q"
      />
      <button type="submit" className="bg-neutral-50 text-neutral-500
rounded-lg p-2">
        <SearchIcon />
      </button>
    </form>
  )
}

```

El siguiente código define un hook personalizado en React llamado useSession, que se utiliza para gestionar el estado de la sesión de usuario en la aplicación.

```

import { Routes } from '@routes'
import type { Session, SessionStatus } from '@types'
import Moralis from 'moralis-v1'
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { $SessionStatus } from '../enum'
// import { getUserById } from '@services/users'

function useSession() {

```

```
const [user, setUser] = useState<Session | null>(null)
const [status, setStatus] = useState<SessionStatus>($SessionStatus.LOADING)
const navigate = useNavigate()

useEffect(() => {
  try {
    const currentSession = Moralis.User.current()

    if (!currentSession?.attributes) {
      setStatus($SessionStatus.UNAUTHENTICATED)
      return
    }

    const { id } = currentSession

    const { sessionToken, email, user_type } = currentSession.attributes

    if (!sessionToken) {
      setStatus($SessionStatus.UNAUTHENTICATED)
      return
    }

    setUser({
      id,
      sessionToken,
      email,
      fullName: '',
      username: '',
      type: user_type,
    })

    // getUserById({ id }).then((response) => {
    //   response.data &&
    //     setUser({
    //       id: currentSession.id,
    //       sessionToken,
    //       email: response.data?.email,
    //       fullName: response.data?.fullName,
    //       username: response.data?.username,
    //       type: response.data?.type
    //     })
    // })
  }
})
```



```

        setStatus($SessionStatus.AUTHENTICATED)
    } catch (error) {
        console.error(error)
        setStatus($SessionStatus.UNAUTHENTICATED_ERROR)
    }
    return () => {
        setUser(null)
    }
}, [])

const signOut = () => {
    Moralis.User.logout()
    navigate(Routes.logIn)
    setStatus($SessionStatus.UNAUTHENTICATED)
}

return {
    user,
    status,
    signOut,
}
}

export default useSession

```

En el siguiente código se muestra el manejo de rutas de la aplicación.

```

import { useBoundStore } from '@store/index'

import { Navigate, Outlet } from 'react-router-dom'

export default function AdminRoutes({ redirectPath = '/log-in' }) {

    const { User, Authenticated, LoadingRefresh } = useBoundStore()

    if (LoadingRefresh) {

        return null

    }

    if (!Authenticated) return <Navigate to={redirectPath} replace={true} />

```

```

// Verificar si User existe y si tiene la propiedad userType

if (User?.user_type && User.user_type !== 'admin') {

    return <Navigate to={redirectPath} replace={true} />

}

return <Outlet />
}

```

```

import { useBoundStore } from '@store/index'
import { Navigate, Outlet } from 'react-router-dom'
export default function IsLogged({ redirectPath = '/log-in' }) {
    const { Authenticated, LoadingRefresh } = useBoundStore()
    if (LoadingRefresh) {
        return null
    }

    if (!Authenticated) {
        return <Navigate to={redirectPath} replace={true} />
    }
    return <Outlet />
}

```

Para la conexión con backend, primero comenzamos ordenando los endpoints como se muestra en el siguiente código.

```

export const Services = {
    auth: {
        signUp: 'createUser',
    },
    users: {
        update: 'updateUser',
        getUserById: 'getUserById',
        getAll: 'getUserOrDriver?type=user',
    },
    table: {
        find: 'findInTable',
    },
} as const

```

En el siguiente fragmento de código se muestra como hacemos el llamado a cada endpoint relacionado con los usuarios.

```
export const signUpUser = async ({ email, password, fullName, username }:
SignUp) => {
  try {
    const response = await Moralis.Cloud.run(Services.auth.signUp, {
      objectData: {
        email,
        password,
        username,
        fullName,
      },
    })

    if (response.status === 'error') {
      return {
        error: 'Ha ocurrido un error',
      }
    }

    return {
      success: 'Registro exitoso',
    }
  } catch (error) {
    console.error(error)
    return {
      error: 'Ha ocurrido un error',
    }
  }
}
```

En la siguiente imagen se muestra cómo gestionamos estados y acciones con Zustand.

```

import type { User } from '@types'

export type UserState = {
  User: User | null
  Authenticated: boolean
  LoadingUser: boolean
  LoadingRefresh: boolean
  LoadingUserEdit: boolean
}

export type UserActions = {
  setUser: (user: User) => void
  setAuthenticated: (value: boolean) => void
  setLoadingUser: (value: boolean) => void
  setLoadingRefresh: (value: boolean) => void
  setLoadingUserEdit: (value: boolean) => void
}

// biome-ignore lint/suspicious/noExplicitAny: <explanation>
export const useUserStore = (set: any) => ({
  User: null,
  ⚡Authenticated: false,
  LoadingUser: false,
  LoadingRefresh: true,
  UserById: null,
  LoadingUserEdit: false,

  setLoadingUser: (value: boolean) => set({ LoadingUser: value }),
  setUser: (user: User) => set(() => ({ User: user })),
  setAuthenticated: (value: boolean) => set({ Authenticated: value }),
  // biome-ignore lint/suspicious/noExplicitAny: <explanation>
  setUserById: (user: any) => set(() => ({ UserById: user })),
  setLoadingUserEdit: (value: boolean) => set({ LoadingUserEdit: value }),
  setLoadingRefresh: (value: boolean) => set({ LoadingRefresh: value }),
})

```

[Ver imagen](#)

Gracias a la adopción de la arquitectura Screaming Architecture en el lado del Frontend se garantiza la claridad, mantenibilidad y escalabilidad del código. Esta arquitectura asegura que la estructura de nuestro código grite su intención, haciendo que cada proyecto sea más fácil de entender y trabajar para cualquier desarrollador.